

Airport Property Management System

API Document

CONTENT

1. Overview
2. Base URL & Authentication
3. API Endpoints
4. Response Formats
5. Error Handling

1. Overview

The APMS Backend provides a RESTful API for managing airport duty-free retail operations, including:

- Administrative authentication
- Parent corporation management
- Retail shop profiling
- Property infrastructure tracking
- Lease agreements
- Sales transactions
- Financial reporting

Tech Stack:

- Runtime: Node.js
- Framework: Express.js (v5.x)
- Database: MySQL via mysql2
- Authentication: JWT (JSON Web Tokens)
- CORS Support: Enabled

2. Base URL & Authentication

Base URL: `http://localhost:5000`

Authentication

Most endpoints (except login) require JWT authentication via the Authorization header:

```
Authorization: Bearer <JWT_TOKEN>
```

Tokens are valid for **1 hour** and should be included in request headers.

3. API Endpoints

3.1. Authentication Router (/api/auth)

Login

POST /api/auth/login

Authenticates administrative user credentials and returns a JWT token.

Request Body:

```
{  
  "username": "admin",  
  "password": "password123"  
}
```

Response (Success - 200):

```
{  
  "message": "Login successful!",  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Response (Error - 401):

```
{  
  "message": "Username is incorrect!"  
}
```

OR

```
{  
  "message": "Password is incorrect!"  
}
```

3.2. Companies Router (/api/companies)

Manages parent corporation records and operational data.

Get All Companies

GET /api/companies

Retrieves all registered parent corporations in descending order.

Response (200):

```
[  
  {  
    "companyID": 1,  
    ...  
  },  
  ...  
]
```

```
"companyName": "Luxury Retail Co.",  
  
"address": "Terminal 3, Zone A",  
  
"contact": "+1-555-0100",  
  
"email": "info@luxuryretail.com",  
  
"Status": "Active"  
  
},  
  
{  
  
  "companyID": 2,  
  
  "companyName": "Premium Goods Ltd.",  
  
  "address": "Terminal 1, Zone B",  
  
  "contact": "+1-555-0200",  
  
  "email": "contact@premiumgoods.com",  
  
  "Status": "Active"  
  
}  
  
]
```

Register New Company

POST /api/companies

Adds a new parent corporation to the system.

Request Body:

```
{  
  
  "companyName": "New Retail Corp",  
  
  "address": "Terminal 2, Zone C",  
  
  "contact": "+1-555-0300",  
  
  "email": "hello@newretail.com"  
  
}
```

Response (200):

```
{  
  
  "message": "Company registered successfully!"  
  
}
```

Update Company

PUT /api/companies/:id

Updates an existing company's information.

Request Body:

```
{  
  "companyName": "Updated Retail Corp",  
  "address": "Terminal 2, Zone C",  
  "contact": "+1-555-0400",  
  "email": "updated@retail.com",  
  "Status": "Active"  
}
```

Response (200):

```
{  
  "message": "Company details updated successfully!"  
}
```

Delete Company

DELETE /api/companies/:id

Removes a company record from the system.

Response (200):

```
{  
  "message": "Company deleted successfully!"  
}
```

3.3. Shops Router (/api/shops)

Manages individual retail tenant shop profiles and configurations.

Get All Shops

GET /api/shops

Retrieves all shops with company associations in descending order.

Response (200):

```
[  
  {  
    "shopId": 1,  
    "shopName": "Premium Watch Store",  
    "companyId": 1,  
    ...  
  },  
  ...  
]
```

```
"companyName": "Luxury Retail Co.",  
  
"contact": "+1-555-1000",  
  
"email": "watches@luxury.com",  
  
"integrationType": "POS_Integration",  
  
"lastDocCount": 150,  
  
"Status": "Active"  
  
}  
  
]
```

Register New Shop

POST /api/shops

Adds a new retail shop to the system.

Request Body:

```
{  
  
"shopName": "Perfume Paradise",  
  
"companyId": 1,  
  
"contact": "+1-555-1100",  
  
"email": "perfume@luxury.com",  
  
"integrationType": "Manual_Entry",  
  
"lastDocCount": 0  
  
}
```

Response (200):

```
{  
  
"message": "Shop registered successfully!"  
  
}
```

Update Shop

PUT /api/shops/:id

Updates an existing shop's information and configuration.

Request Body:

```
{  
  
"shopName": "Perfume Paradise Updated",  
  
"companyId": 1,  
  
}
```

```
"contact": "+1-555-1150",  
"email": "updated@perfume.com",  
"integrationType": "POS_Integration",  
"lastDocCount": 200,  
"Status": "Active"  
}
```

Response (200):

```
{  
  
  "message": "Shop updated successfully!"  
}
```

Delete Shop

DELETE /api/shops/:id

Removes a shop record from the system.

Response (200):

```
{  
  
  "message": "Shop deleted successfully!"  
}
```

3.4. Properties Router (/api/properties)

Tracks terminal structural properties, dimensions, and spatial allocation.

Get All Properties

GET /api/properties

Retrieves all property records in descending order.

Response (200):

```
[  
  
  {  
  
    "propertyID": 1,  
    "PropertyName": "Zone A - Premium Display",  
    "sqft": 1500,  
    "locationId": "T3-A1",  
    "leased": "Yes",
```



```
"proposedProperty": false,  
"chartColor": "#FF5733"  
}  
]
```

Get Unique Locations

GET /api/properties/unique-locations

Retrieves all distinct location IDs in the property database.

Response (200):

```
[  
  "T1-A",  
  "T1-B",  
  "T2-A",  
  "T3-A1",  
  "T3-B"  
]
```

Add New Property

POST /api/properties/add

Registers a new property in the system.

Request Body:

```
{  
  "PropertyName": "Zone B - Standard Display",  
  "sqft": 1200,  
  "locationId": "T2-B1",  
  "leased": "No",  
  "proposedProperty": false,  
  "chartColor": "#3366CC"  
}
```

Response (200):

```
{  
  "message": "Property registered successfully!"  
}
```

Update Property

PUT /api/properties/:id

Updates an existing property's information.

Request Body:

```
{  
  "PropertyName": "Zone A - Premium Display (Upgraded)",  
  "sqft": 1800,  
  "locationId": "T3-A1",  
  "leased": "Yes",  
  "proposedProperty": false,  
  "chartColor": "#FF6B6B"  
}
```

Response (200):

```
{  
  "message": "Property updated successfully!"  
}
```

Delete Property

DELETE /api/properties/:id

Removes a property record.

Response (200):

```
{  
  "message": "Property deleted successfully!"  
}
```

3.5. Agreements Router (/api/agreements)

Manages corporate lease contracts, rental rates, and agreement matrices.

Get All Agreements

GET /api/agreements

Retrieves all lease agreements with company and property details.

Response (200):

```
[  
  
  {  
  
    "agreementId": 1,  
  
    "shopId": 1,  
  
    "companyId": 1,  
  
    "companyName": "Luxury Retail Co.",  
  
    "propertyID": 1,  
  
    "PropertyName": "Zone A - Premium Display",  
  
    "startDate": "2024-01-01",  
  
    "endDate": "2025-12-31",  
  
    "minimumSTOLevel": 50000,  
  
    "confeeRate": 15,  
  
    "BankDepositAmount": "100000",  
  
    "agreementStatus": "Active",  
  
    "locationId": "T3-A1",  
  
    "stoCurrency": "LKR"  
  
  }  
  
]
```

Create New Agreement

POST /api/agreements

Registers a new lease agreement.

Request Body:

```
{  
  
  "shopId": 1,  
  
  "companyId": 1,  
  
  "propertyID": 2,  
  
  "startDate": "2024-06-01",  
  
  "endDate": "2026-05-31",  
  
  "minimumSTOLevel": 75000,  
  
  "confeeRate": 18,  
  
  "BankDepositAmount": "150000",  
  
  "agreementStatus": "Active",  
  
  "locationId": "T3-B",  
  
}
```

```
"stoCurrency": "LKR"
```

```
}
```

Response (200):

```
{
```

```
"message": "Agreement registered successfully!"
```

```
}
```

Update Agreement

PUT /api/agreements/:id

Updates an existing lease agreement.

Request Body:

```
{
```

```
"shopId": 1,
```

```
"companyId": 1,
```

```
"propertyID": 2,
```

```
"startDate": "2024-06-01",
```

```
"endDate": "2026-05-31",
```

```
"minimumSTOLevel": 80000,
```

```
"confeeRate": 20,
```

```
"BankDepositAmount": "160000",
```

```
"agreementStatus": "Active",
```

```
"locationId": "T3-B",
```

```
"stoCurrency": "LKR"
```

```
}
```

Response (200):

```
{
```

```
"message": "Agreement updated successfully!"
```

```
}
```

Delete Agreement

DELETE /api/agreements/:id

Removes a lease agreement.

Response (200):

```
{  
  "message": "Agreement deleted successfully!"  
}
```

Get Locations List

GET /api/agreements/locations-list

Retrieves all distinct locations for agreements.

Response (200):

```
[  
  { "locationId": "T1-A" },  
  { "locationId": "T2-B" },  
  { "locationId": "T3-A1" }  
]
```

3.6. Sales Router (/api/sales)

Captures, registers, and manages terminal transaction data.

Get All Shops (for Sales)

GET /api/sales/shops

Retrieves shop list for transaction assignment.

Response (200):

```
[  
  {  
    "shopId": 1,  
    "shopName": "Premium Watch Store",  
    "companyId": 1  
  }  
]
```

Get All Companies (for Sales)

GET /api/sales/companies

Retrieves company list for transaction association.

Response (200):

```
[
{
  "companyId": 1,
  "companyName": "Luxury Retail Co."
}
]
```

Get All Locations (for Sales)

GET /api/sales/locations

Retrieves location list for transaction filtering.

Response (200):

```
[
{
  "locationId": "T3-A1",
  "locationName": "Terminal 3 - Zone A1"
}
]
```

Get Recent Transactions

GET /api/sales/recent

Fetches the 50 most recent sales transactions with full details.

Response (200):

```
[
{
  "dayId": 1001,
  "shopId": 1,
  "CONCESSIONAR_NAME": "Luxury Retail Co.",
  "shopName": "Premium Watch Store",
  "LOCATION": "T3-A1",
  "INVOICE_NUMBER": "INV-20240615-001",
  "TRANSACTION_DATE": "2024-06-15",
  "TRANSACTION_TIME": "14:30:45",
  "PRODUCT_NAME": "Luxury Wristwatch",
  "PRODUCT_CATEGORY": "Accessories",
```

```
"PRODUCT_SUB_CATEGORY": "Watches",  
  
"BRAND_NAME": "Rolex",  
  
"QUANTITY": 1,  
  
"UNIT_PRICE": 5000,  
  
"TOTAL_AMOUNT_BEFORE_DISCOUNT": 5000,  
  
"DISCOUNT_AMOUNT": 0,  
  
"DISCOUNT_TYPE": "None",  
  
"TOTAL_AMOUNT_AFTER_DISCOUNT": 5000,  
  
"SALES_TAX_PERCENTAGE": 10,  
  
"SALES_TAX": 500,  
  
"NET_SALES": 5500,  
  
"MINUS_TAX": 5000,  
  
"PAYMENT_METHOD": "Card",  
  
"CURRENCY": "USD",  
  
"ACTUAL_PAYMENT_CURRENCY_TYPE": "USD",  
  
"TRANSACTION_TYPE": "SAL",  
  
"VOID_CANCELTION_TYPE": "NONE",  
  
"FLIGHT": "AA-123",  
  
"FLIGHT_DATE_TIME": "2024-06-15 16:00",  
  
"AIRPORT_ORG": "JFK",  
  
"AIRPORT_DES": "LHR",  
  
"PASSENGER_ID_NAME": "John Doe",  
  
"NATIONALITY": "USA",  
  
"PASSPORT_ID": "J12345678",  
  
"NATIONAL_ID": "N/A",  
  
"BIRTHDATE": "1980-05-15",  
  
"GENDER": "M",  
  
"dbn": "LIVE",  
  
"tillNo": "Till-01",  
  
"is_duplicate": 0  
  
}  
  
]
```

Add Transaction(s)

POST /api/sales/add

Adds one or multiple sales transactions to the system.

Request Body (Single Transaction):

```
{
  "shopId": 1,
  "CONCESSIONAR_NAME": "Luxury Retail Co.",
  "LOCATION": "T3-A1",
  "INVOICE_NUMBER": "INV-20240620-001",
  "TRANSACTION_DATE": "2024-06-20",
  "TRANSACTION_TIME": "15:45:00",
  "PRODUCT_NAME": "Luxury Perfume",
  "PRODUCT_CATEGORY": "Beauty",
  "PRODUCT_SUB_CATEGORY": "Fragrances",
  "BRAND_NAME": "Chanel",
  "QUANTITY": 2,
  "UNIT_PRICE": 150,
  "TOTAL_AMOUNT_BEFORE_DISCOUNT": 300,
  "DISCOUNT_AMOUNT": 30,
  "DISCOUNT_TYPE": "Percentage",
  "TOTAL_AMOUNT_AFTER_DISCOUNT": 270,
  "SALES_TAX_PERCENTAGE": 10,
  "SALES_TAX": 27,
  "NET_SALES": 297,
  "MINUS_TAX": 270,
  "PAYMENT_METHOD": "Card",
  "CURRENCY": "USD",
  "ACTUAL_PAYMENT_CURRENCY_TYPE": "USD",
  "TRANSACTION_TYPE": "SAL",
  "VOID_CANCELATION_TYPE": "NONE",
  "FLIGHT": "BA-456",
  "FLIGHT_DATE_TIME": "2024-06-20 18:00",
  "AIRPORT_ORG": "LHR",
```



```
"AIRPORT_DES": "CDG",  
"PASSENGER_ID_NAME": "Jane Smith",  
"NATIONALITY": "UK",  
"PASSPORT_ID": "G87654321",  
"GENDER": "F",  
"dbn": "LIVE",  
"tillNo": "Till-02"  
}
```

Request Body (Batch Transactions):

```
[  
  { /* transaction 1 */ },  
  { /* transaction 2 */ }  
]
```

Response (Success - 200):

```
{  
  "message": "Successfully synced 2 out of 2 transactions!",  
  "errors": []  
}
```

Response (Partial Success - 200):

```
{  
  "message": "Successfully synced 1 out of 2 transactions!",  
  "errors": [  
    "shopId #999 not found"  
  ]  
}
```

Update Transaction

PUT /api/sales/update/:id

Updates an existing transaction record.

Request Body: (Same as POST /api/sales/add)

Response (200):

```
{  
  "message": "Transaction Log ID #1001 updated successfully!"  
}
```

Delete Transaction

DELETE /api/sales/delete/:id

Removes a transaction record.

Response (200):

```
{
  "message": "Transaction ID #1001 deleted!"
}
```

3.7. Reports Router (/api/sales)

Provides financial analytics and transaction reporting.

Get Daily Sales Report

GET /api/sales/report

Fetches sales data for a specific date.

Query Parameters:

- **date (required):** Report date in format YYYY-MM-DD

Example: /api/sales/report?date=2024-06-15

Response (200):

```
[
  {
    "minLimit": 50000,
    "companyName": "Luxury Retail Co.",
    "shopName": "Premium Watch Store",
    "saleAmount": 5500,
    "currency": "USD"
  }
]
```

Get Monthly Sales Report

GET /api/sales/report-monthly

Fetches aggregated sales data for a specific month.

Query Parameters:

- month (required): Report month in format YYYY-MM

Example: /api/sales/report-monthly?month=2024-06

Response (200):

```
[
  {
    "minLimit": 50000,
    "companyName": "Luxury Retail Co.",
    "shopName": "Premium Watch Store",
    "saleAmount": 125000,
    "currency": "USD"
  }
]
```

Get Shop Monthly Summary

GET /api/sales/report-shop-monthly-summary

Fetches detailed summary for a specific shop in a specific month.

Query Parameters:

- shopId (required): Shop ID
- month (required): Report month in format YYYY-MM

Example: /api/sales/report-shop-monthly-summary?shopId=1&month=2024-06

Response (200):

```
[
  {
    "reportMonth": "2024-06",
    "agreementId": 1,
    "companyName": "Luxury Retail Co.",
  }
]
```

```
"shopName": "Premium Watch Store",  
"lastDate": "2024-06-30",  
"lastInvoiceNo": "INV-20240630-050",  
"saleAmount": 125000  
}  
]
```

Get Monthly Invoice Count Report

GET /api/sales/report-monthly-invoice-count

Fetches invoice count and sales data grouped by shop for a specific month.

Query Parameters:

- month (required): Report month in format YYYY-MM

Example: /api/sales/report-monthly-invoice-count?month=2024-06

Response (200):

```
[  
  {  
    "shopId": 1,  
    "shopName": "Premium Watch Store",  
    "reportYear": "2024",  
    "reportMonth": "6",  
    "invoiceCount": 50,  
    "saleAmount": 125000,  
    "currency": "USD",  
    "minLimit": 50000,  
    "companyName": "Luxury Retail Co."  
  }  
]
```

4. Response Formats

Success Response (200 OK)

```
{
  "message": "Operation successful!",
  "data": {}
}
```

Error Response (4xx/5xx)

```
{
  "message": "Error description"
}
```

Common HTTP Status Codes

Code	Meaning
200	OK - Request successful
400	Bad Request - Invalid input parameters
401	Unauthorized - Invalid/expired JWT token or credentials
404	Not Found - Resource not found
500	Internal Server Error - Database or server error

5. Error Handling

Database Errors

When database operations fail, the API returns a 500 status with an error message:

```
{
  "message": "Database error!"
}
```

```
}
```

Validation Errors

When required fields are missing:

```
{  
  "message": "Required attributes (shopId, INVOICE_NUMBER, NET_SALES) are missing!"  
}
```

Missing Query Parameters

```
{  
  "message": "Please provide a valid report date!"  
}
```

Setup & Configuration

Environment Variables (.env)

```
PORT=5000  
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=  
JWT_SECRET=your_custom_secure_jwt_secret_key_string
```

Running the Server

```
cd backend  
npm install  
node server.js
```

The server will start on <http://localhost:5000>